

ESPECIFICAÇÃO TÉCNICA E DE ARQUITETURA

Plataforma Telemetria e Saúde de Ativos Industriais (OEM Asset Care)

Documento Oficial de Engenharia — Versão 1.0

Gerência de Engenharia e Projetos

26 de setembro de 2026

Resumo

Este documento estabelece as diretrizes técnicas, arquiteturais e funcionais obrigatórias para a implementação da plataforma de telemetria e diagnóstico remoto para equipamentos industriais. É direcionado à equipe de desenvolvimento e serve como a única fonte da verdade para o ciclo de vida de engenharia do sistema.

Índice

1	Instruções da Gerência & Diretrizes de Engenharia	3
1.1	Perfil do Projeto e Expectativas	3
1.2	Princípios de Desenvolvimento	3
2	Arquitetura Geral e Topologia de Containers	3
2.1	Visão de Alto Nível	3
2.2	Estrutura Oficial de Diretórios (Monorepo)	3
3	Especificação Detalhada dos Microserviços	4
3.1	Componente Auxiliar: IoT Device Simulator (<code>iot-simulator</code>)	4
3.2	Microserviço 1: Ingestion & Telemetry Worker (<code>ingestion-worker</code>)	4
3.3	Microserviço 2: Core API & Management (<code>core-api</code>)	5
3.4	Microserviço 3: Alert & Stream Engine (<code>alert-engine</code>)	5
3.5	Microserviço 4: Realtime Gateway (<code>realtime-gateway</code>)	6
4	Modelo de Dados e Estratégia de Persistência	6
4.1	PostgreSQL + Extensão TimescaleDB	6

1 Instruções da Gerência & Diretrizes de Engenharia

1.1 Perfil do Projeto e Expectativas

Como esta é a primeira implementação deste porte conduzida por esta equipe, a aderência estrita às especificações contidas neste documento é obrigatória. O projeto deve demonstrar maturidade arquitetural, resiliência no processamento de dados e separação clara de responsabilidades.

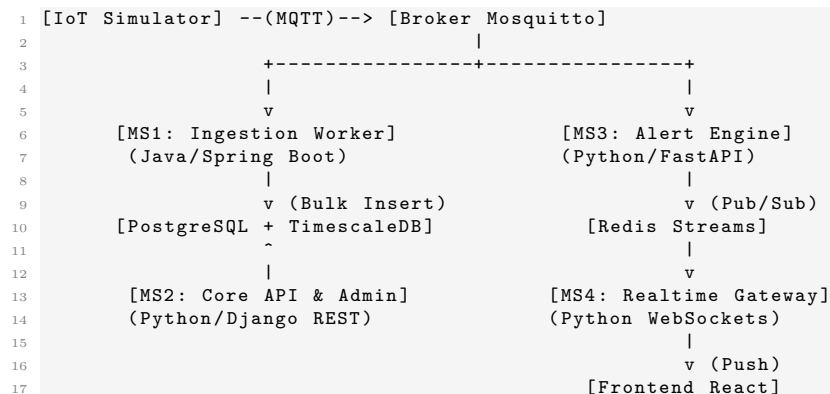
1.2 Princípios de Desenvolvimento

- **SOLID & DRY:** Cada classe, módulo ou método deve possuir uma única responsabilidade. Código duplicado não será aceito na revisão.
- **Monorepo Organizacional:** O código fonte de todos os microserviços e componentes de simulação deve residir no mesmo repositório Git, viabilizando execução simples via Docker Compose.
- **Tratamento Interno de Exceções:** Nenhum erro não tratado deve interromper o ciclo de vida dos containers. Falhas de parse, queda de banco de dados ou perda de conexões TCP devem ser capturadas e tratadas no nível local do microserviço.
- **Isolamento Multi-Tenant:** Toda e qualquer consulta, gravação de métrica ou emissão de alerta deve carregar rigorosamente o `tenant_id` da organização proprietária da máquina.

2 Arquitetura Geral e Topologia de Containers

2.1 Visão de Alto Nível

O sistema utiliza uma arquitetura baseada em eventos (*Event-Driven Microservices*) containerizada via Docker, projetada para execução local em ambiente de desenvolvimento e pronta para migração simplificada para a infraestrutura AWS (ECS/RDS/ElastiCache).



Listing 1: Arquitetura da Solução e Fluxo de Dados

2.2 Estrutura Oficial de Diretórios (Monorepo)

A estrutura do repositório deve seguir rigorosamente a disposição abaixo:

```
1 meu-projeto-iot/
2   docker-compose.yml
3   .env.example
4   README.md
5   iot-simulator/           # Python (Simulador de Maquinas e Anomalias)
6   ingestion-worker/        # Java / Spring Boot (Ingestao de Altissima Vazao)
```

```

7     core-api/                # Python / Django REST + Admin (Gestao Relacional)
8     alert-engine/           # Python / FastAPI (Processamento de Regras)
9     realtime-gateway/       # Python / FastAPI WebSockets (Notificacoes)

```

Listing 2: Diretório Raiz do Projeto

3 Especificação Detalhada dos Microserviços

3.1 Componente Auxiliar: IoT Device Simulator (iot-simulator)

Linguagem: Python (Asyncio)

Objetivo: Emular a frota de equipamentos enviando mensagens MQTT periódicas e injetar falhas operacionais sob demanda para validação dos alarmes do sistema.

1. `machine_emulator.start_simulation()`

- **Objetivo:** Inicializar o ciclo assíncrono de simulação criando conexões ativas para cada máquina virtualizada.
- **Conexões:** Invoca `data_generator.generate_payload()`.

2. `data_generator.generate_payload()`

- **Objetivo:** Gerar leituras verossímeis de temperatura, horímetro e vibração, permitindo induzir anomalias críticas.
- **Conexões:** Envia a estrutura gerada para `machine_emulator.publish_mqtt()`.

3. `simulator_error_handler.handle_publish_error()`

- **Objetivo:** Tratar perdas pontuais de conexão com o broker aplicando retentativa exponencial (*backoff*).
- **Conexões:** Notifica o log e reconecta o emulador.

3.2 Microserviço 1: Ingestion & Telemetry Worker (ingestion-worker)

Linguagem: Java (Spring Boot)

Objetivo: Consumir os tópicos de telemetria com alta eficiência de I/O, validar a integridade dos esquemas e realizar gravações em lote (*bulk insert*) no TimescaleDB.

1. `TelemetryConsumer.subscribeToBroker()`

- **Objetivo:** Escutar o barramento MQTT/AMQP e receber payloads brutos de telemetria.
- **Conexões:** Direciona o payload para `PayloadValidator.validateSchema()`.

2. `PayloadValidator.validateSchema()`

- **Objetivo:** Validar os campos obrigatórios (`tenant_id`, `machine_id`, `timestamp`, `metrics`) e tipagem do JSON.
- **Conexões:** Se válido, envia para `TelemetryBatchBuffer.add()`; se inválido, invoca `IngestionExceptionHandler.h`

3. `TelemetryBatchBuffer.add()` e `flush()`

- **Objetivo:** Agrupar métricas em buffer de memória até atingir 100 registros ou 2 segundos, disparando a inserção massiva.
- **Conexões:** Invoca `TimescaleRepository.bulkInsert()`.

4. `TimescaleRepository.bulkInsert()`

- **Objetivo:** Executar a instrução SQL COPY ou INSERT em lote diretamente na Hypertable do TimescaleDB.
- **Conexões:** Retorna o status de confirmação (ACK) para o consumidor.

5. `IngestionExceptionHandler.handleDatabaseError()`

- **Objetivo:** Capturar indisponibilidades temporárias do banco e mover a mensagem corrompida/rejeitada para uma fila secundária de análise (*Dead Letter Queue*).

3.3 Microserviço 2: Core API & Management (core-api)

Linguagem: Python (Django REST Framework)

Objetivo: Fornecer o painel administrativo back-office, cadastro de clientes, vínculo de equipamentos e definição de limites de alarme.

1. `tenants.views.create_tenant()`

- **Objetivo:** Processar o registro de novos clientes garantindo o isolamento relacional e credenciais operacionais.
- **Conexões:** Persiste dados no PostgreSQL via ORM.

2. `machines.views.register_machine()`

- **Objetivo:** Cadastrar equipamentos, associando identificadores de hardware aos limites operacionais de fábrica.
- **Conexões:** Salva no PostgreSQL e aciona `sync_publisher.publish_machine_created()`.

3. `alert_configs.views.update_threshold()`

- **Objetivo:** Alterar parâmetros operacionais tolerados (ex: temperatura máxima ajustada para $75^{\circ}C$).
- **Conexões:** Atualiza o banco e invoca `sync_publisher.publish_threshold_updated()`.

4. `core.exceptions.custom_exception_handler()`

- **Objetivo:** Padronizar todas as respostas de erro da API no formato JSON corporativo `{"error": "código", "message": "detalhes"}`.

3.4 Microserviço 3: Alert & Stream Engine (alert-engine)

Linguagem: Python (FastAPI / Asyncio)

Objetivo: Processar em tempo real os fluxos de eventos do Redis Streams e avaliar desvios operacionais.

1. `redis_consumer.read_telemetry_stream()`

- **Objetivo:** Ler continuamente o fluxo de dados em tempo real gravado pelo broker no Redis Streams.
- **Conexões:** Envia cada leitura para `rule_evaluator.evaluate()`.

2. `rules_cache.get_machine_rules()`

- **Objetivo:** Buscar do cache em memória (Redis) os limites ativos configurados para a máquina sem sobrecarregar o banco relacional.

3. `rule_evaluator.evaluate()`

- **Objetivo:** Comparar a telemetria lida com os limites permitidos.
- **Conexões:** Se ultrapassado, chama `alert_publisher.trigger_alert()`.

4. `alert_publisher.trigger_alert()`

- **Objetivo:** Formatar a notificação de violação e publicar no canal Pub/Sub do Redis para consumo do gateway de tempo real.

3.5 Microserviço 4: Realtime Gateway (realtime-gateway)

Linguagem: Python (FastAPI WebSockets)

Objetivo: Manter conexões TCP ativas com os navegadores/apps dos clientes e entregar atualizações instantâneas de falhas.

1. `manager.connect()` e `jwt_validator.decode_token()`

- **Objetivo:** Estabelecer o handshake WebSocket, validar a autenticidade do token JWT e extrair o `tenant_id`.

2. `manager.register_session()`

- **Objetivo:** Agrupar a conexão socket do usuário ativo em salas nomeadas pelo identificador do seu `tenant`.

3. `alert_listener.listen_alerts()` e `manager.broadcast_to_tenant()`

- **Objetivo:** Escutar os alertas emitidos pelo MS3 e transmiti-los em tempo real apenas para as conexões do `tenant` afetado.

4. `ws_error_handler.handle_disconnect()`

- **Objetivo:** Encerrar conexões mortas ou corrompidas e liberar recursos do servidor instantaneamente.

4 Modelo de Dados e Estratégia de Persistência

4.1 PostgreSQL + Extensão TimescaleDB

A persistência da solução é unificada na mesma instância do PostgreSQL para reduzir os custos operacionais.

- **Tabelas Relacionais Tradicionais:** Armazenam `tenants`, `users`, `machines` e `alert_thresholds`.
- **Hypertables (Séries Temporais):** A tabela `telemetry_data` é convertida em uma Hypertable particionada por tempo (`timestamp`) e `machine_id`.

```
1 CREATE TABLE telemetry_data (  
2     timestamp TIMESTAMPTZ NOT NULL,  
3     tenant_id UUID NOT NULL,  
4     machine_id VARCHAR(50) NOT NULL,  
5     metrics JSONB NOT NULL  
6 );  
7  
8 -- Converte a tabela padrao em uma Hypertable otimizada para tempo  
9 SELECT create_hypertable('telemetry_data', 'timestamp');  
10  
11 -- Cria indice composto para consultas rapidas por cliente e maquina  
12 CREATE INDEX idx_tenant_machine_time  
13 ON telemetry_data (tenant_id, machine_id, timestamp DESC);
```

Listing 3: Script DDL da Tabela de Telemetria (TimescaleDB)